



PROCESADORES DE LENGUAJE - P2

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

Cuestiones abiertas:

2.1 Observa el resultado de diversas expresiones como:

$$2*3+1 = 8 \quad \text{X}$$

$$1+2*3 = 7 \quad \checkmark$$

$$2+3*1 = 5 \quad \checkmark$$

$$1*3+2 = 5 \quad \checkmark$$

$$1-1-1 = 1 \quad \text{X}$$

$$1-1-1-1 = 0 \quad \text{X}$$

$$1-1-1-1-1 = 1 \quad \text{X}$$

$$1-1-1-1-1-1 = 0 \quad \text{X}$$

$$1-2-3-4-5 = 3 \quad \text{X}$$

¿Sabes determinar a qué se deben los resultados obtenidos?

Esto ocurre porque al ser un autómatas a pila el que resuelve, lo hace de manera recursiva y esto implica que va cogiendo expresiones de derecha a izquierda. Por ejemplo:

La expresión $2*3+1$ la realiza de esta manera:

Axioma $\rightarrow$ Expresión(2) Operando(+) **Expresión(3+1)**

Expresión(3+1) $\rightarrow$ Expresión(3) Operando(+) Expresión(1) $\rightarrow 4$

Una vez tiene el 4 de la segunda expresión opera el $2+4$ ya que no distingue entre la prioridad de las operaciones por tanto hace $2*4$ y da 8 en lugar de 7.

Y esto es lo que pasa con todos.

Debido a la casualidad que el 2, 3, 4 dan solución correcta pero por que el orden de prioridad de las operaciones es el mismo que el orden que sigue el compilador para resolver la operación

2.2. Existe un problema en la solución desarrollada hasta ahora:

1 2 3 + 2 1 Expresión=144.000000

¿A qué se debe esto? ¿Qué soluciones se te ocurren? Analiza el código del analizador léxico (yylex).

El error se debe a que la función `yylex` identifica como correcto los espacios en blanco que hay entre dígitos. Podríamos modificar la función para que en cuanto identifique el primer espacio en blanco entre dígitos antes de llegar al operador devuelva una 'Syntax error'. Una solución posible sería mediante la sustitución del `while` por un `if` y pare de iterar devolviendo el mensaje de error pertinente..

2.3. Hay un pequeño fallo tal como está definida la expresión. ¿Sabes en qué casos aparece? ¿Y a qué es debido? Prueba diversos tipos de expresiones.

El problema que hay viene dado por expresiones que a la vista tengan sentido pero el programa devuelve un error. Estas expresiones son las que contengan casos como:

)” o (+-) que a la vista tienen solución pero nuestra gramática no entiende, ya que no las estamos teniendo en cuenta. Para solucionarlo hemos establecido reglas para cada nivel de preferencia:

expresion maneja “+” y “-”.

termino maneja “*” y “/”.

factor maneja paréntesis y valores (operando).

Así, por ejemplo, “(1+1)*3” primero calcula el contenido entre paréntesis (1+1), después usa el resultado como un factor para la multiplicación por 3. Esto evita los problemas típicos de precedencia y orden de evaluación.

Pasando de la primera foto a la segunda

```
expresion:  operando      { $$ = $1 ; }
            | operando '+' expresion { $$ = $1 + $3 ; }
            | operando '-' expresion { $$ = $1 - $3 ; }
            | operando '*' expresion { $$ = $1 * $3 ; }
            | operando '/' expresion { $$ = $1 / $3 ; }
            | '(' expresion ')' { $$ = $2 ; }
            ;
```

→

```
expresion:
    expresion '+' termino { $$ = $1 + $3; }
  | expresion '-' termino { $$ = $1 - $3; }
  | termino                { $$ = $1; }
  ;

termino:
    termino '*' factor { $$ = $1 * $3; }
  | termino '/' factor { $$ = $1 / $3; }
  | factor              { $$ = $1; }
  ;

factor:
    '(' expresion ')' { $$ = $2; }
  | operando          { $$ = $1; }
  ;
```

2.4. Se ha diseñado la definición de número como:

```
numero:      digito      { $$ = $1 ; pot = 1 ; }
            | digito numero { pot *= 10 ; $$ = $1 * pot + $2 ; }
            ;
```

Usando una estructura recursiva similar a la que habíamos empleado inicialmente para axioma (Caso Base, Caso Base + Recursividad):

```
axioma:      expresion '\n' { printf ("Expresion=%lf\n", $1) ; }
            | expresion '\n' { printf ("Expresion=%lf\n", $1) ; } axioma
            ;
```

Sin embargo, en el caso de axioma se tuvo que descartar por los problemas que causaba. ¿Sabes decir por qué vale en el primer caso y no en el segundo?

En “número” la recursividad es clara y no genera ambigüedad: cada dígito se añade al final sin conflictos. En cambio, en “axioma” la recursión causaba problemas de reentrada y conflictos en la gramática, debiendo reestructurarse para evitar ambigüedades.